

PetCare AI Companion

Proyecto final del modulo Desarrollo Vibe Coding

Autor: Alvaro Ferrer

Repositorio: <https://github.com/alvaroferrer1/PetCare-.git>

1. Descripcion general del sistema

PetCare AI Companion es una aplicacion movil desarrollada con Flutter para organizar el cuidado diario de perros y gatos. La aplicacion permite centralizar en un unico lugar la informacion principal de cada mascota: datos generales, vacunas, visitas veterinarias, medicacion, alimentacion, higiene, notas de salud, peso, documentos, contactos veterinarios, recordatorios y consultas de seguridad alimentaria.

El objetivo del proyecto es resolver un problema habitual: muchas personas guardan la informacion de sus mascotas de forma dispersa, en notas del movil, conversaciones, papeles veterinarios o simplemente de memoria. Esto puede provocar olvidos, perdida de informacion importante o dificultad para preparar una visita al veterinario.

La solucion propuesta es una app funcional, estructurada y demostrable, que combina persistencia de datos, una interfaz movil, integracion con servicios externos y un asistente de IA responsable que ayuda a resumir la informacion registrada.

2. Problema que resuelve

La app resuelve la desorganizacion del cuidado de mascotas. En concreto ayuda a:

- Recordar vacunas, medicacion, revisiones y otros cuidados pendientes.
- Consultar rapidamente el historial de una mascota.
- Registrar sintomas, apetito, energia y observaciones de salud.
- Preparar una visita veterinaria con informacion ordenada.
- Revisar si un alimento puede ser seguro, de precaucion, toxico o desconocido.
- Analizar productos reales cruzando ingredientes con una base de riesgos.
- Tener a mano contactos veterinarios y datos utiles en caso de emergencia.

3. Para que sirve

PetCare AI Companion sirve como centro de control para el cuidado de mascotas. Esta pensada para duenos de perros y gatos que quieren tener el historial de salud, recordatorios y datos importantes siempre accesibles.

La app no pretende diagnosticar enfermedades ni sustituir al veterinario. Su papel es organizar informacion, ayudar a detectar prioridades y facilitar que el usuario llegue mejor preparado a una consulta profesional.

4. Vision global de la solucion

El sistema funciona alrededor de tres partes principales:

- Aplicacion movil Flutter: muestra las pantallas, formularios, listados, filtros y navegacion.
- Estado y logica de aplicacion: un controlador central coordina los datos, las acciones del usuario y la comunicacion con servicios externos.
- Base de datos Supabase: guarda usuarios, mascotas, eventos, notas, contactos, documentos, pesos, consultas de productos y resúmenes de IA.

La aplicacion arranca cargando la configuracion desde el archivo .env. Si Supabase esta configurado, inicializa la conexion con la base de datos. Despues decide que pantalla mostrar:

- Onboarding, si el usuario aun no lo ha completado.
- Login o registro, si no hay sesion iniciada.
- Home, si el usuario ya esta autenticado.

Tambien existe un modo demo para la grabacion del proyecto. Este modo carga mascotas y datos de ejemplo sin depender de crear toda la informacion manualmente durante el video.

5. Herramientas de vibe coding utilizadas

Durante el desarrollo se han usado herramientas de asistencia con IA para avanzar de forma iterativa, revisar ideas, generar codigo, depurar errores, mejorar pantallas, escribir tests y preparar la documentacion del proyecto.

El trabajo se ha realizado siguiendo un enfoque incremental:

- Analizar el problema.
- Dividir la app por funcionalidades.
- Implementar pantallas y modelos.
- Validar con tests.
- Revisar errores de analisis y compilacion.
- Mejorar la experiencia para la demostracion final.

Este enfoque encaja con el modulo porque no solo se ha generado codigo, sino que se ha trabajado con control de versiones, validacion, estructura mantenible y decisiones tecnicas justificadas.

6. Tecnologias empleadas

Las tecnologias principales del proyecto son:

- Flutter: framework principal para construir la aplicacion movil.
- Dart: lenguaje de programacion usado por Flutter.
- Riverpod: gestion de estado de la aplicacion.
- Supabase: backend, autenticacion y base de datos.
- PostgreSQL: base de datos relacional usada por Supabase.

- Row Level Security: seguridad a nivel de fila para proteger datos por usuario.
- Open Pet Food Facts: fuente externa para consultar productos de alimentacion de mascotas.
- Dog CEO API: servicio externo relacionado con razas e imagenes de perros.
- The Cat API: servicio externo relacionado con razas e imagenes de gatos.
- Git y GitHub: control de versiones y repositorio del proyecto.
- Flutter Test: tests unitarios, de logica y de interfaz.

7. Base de datos y persistencia

El proyecto cumple el requisito de persistencia de datos usando Supabase. La estructura de base de datos esta definida en supabase/schema.sql y tambien existen migraciones en la carpeta supabase/migrations.

Tablas principales:

- `profiles`: informacion del usuario.
- `pets`: mascotas registradas.
- `care\_events`: vacunas, visitas veterinarias, medicacion, alimentacion, higiene y otros cuidados.
- `health\_notes`: notas de salud, sintomas, apetito, energia y observaciones.
- `food\_safety\_items`: base curada de alimentos seguros, con precaucion o toxicos.
- `ai\_summaries`: resúmenes generados por IA.
- `product\_checks`: analisis de productos e ingredientes.
- `vet\_contacts`: contactos veterinarios.
- `pet\_documents`: documentos asociados a una mascota.
- `weight\_logs`: registros de peso.

La base de datos incluye claves, relaciones entre tablas y politicas de Row Level Security para que cada usuario solo pueda acceder a sus propios datos.

8. Control de versiones

El proyecto usa Git y GitHub. El repositorio remoto es:

<https://github.com/alvaroferrer1/PetCare-.git>

Ramas usadas:

- `inicio`: rama principal configurada en GitHub.
- `medio`: rama de desarrollo intermedio.

Commits principales del historial:

- `inicio de la app`
- `diseno front y ux pruebas`
- `anade analisis gratuito de productos`
- `proyecto final`

Esto demuestra que el proyecto no se ha trabajado como un unico bloque final, sino mediante avances guardados en control de versiones.

9. Funcionalidades implementadas

9.1 Onboarding

La app comienza con un onboarding de varias paginas. Explica que la aplicacion permite centralizar mascotas, historial, alimentos, productos y asistencia con IA responsable.

El objetivo del onboarding es que el usuario entienda rapidamente para que sirve la app antes de entrar al login.

9.2 Autenticacion

La pantalla de autenticacion permite iniciar sesion o registrarse. Incluye validaciones de email, contrasena y nombre.

Si Supabase esta configurado, la app usa autenticacion real. Para facilitar la demostracion del proyecto, tambien existe un boton de modo demo que carga datos preparados para grabar el video sin depender de introducir toda la informacion manualmente.

9.3 Home

La pantalla principal muestra un resumen global:

- Numero de mascotas.
- Recordatorios pendientes.
- Notas registradas.
- Accesos rapidos a funcionalidades.
- Listado de mascotas.
- Proximos recordatorios.

Desde Home se puede navegar a casi toda la aplicacion.

9.4 Mascotas

El usuario puede crear, editar y eliminar mascotas. Cada mascota tiene datos como nombre, especie, raza, fecha de nacimiento, peso, notas y alergias.

La pantalla de detalle de mascota centraliza su informacion y permite acceder a eventos, notas, timeline, peso, documentos, bienestar, informe veterinario y resumen IA.

9.5 Historial de cuidados

Cada mascota puede tener eventos de cuidado:

- Vacunas.
- Visitas veterinarias.

- Medicacion.
- Alimentacion.
- Higiene.
- Desparasitacion.
- Otros cuidados.

Los eventos tienen estado pendiente, completado o cancelado. La pantalla de detalle permite filtrar el historial por tipo, estado y fecha.

9.6 Notas de salud

La app permite registrar notas de salud con sintomas, estado de animo, apetito, energia y observaciones. Estas notas ayudan a tener contexto antes de una visita veterinaria y tambien alimentan el resumen de IA.

9.7 Recordatorios y calendario

Los eventos pendientes aparecen como recordatorios. Pueden marcarse como completados. Tambien hay una pantalla de calendario mensual para consultar cuidados por fecha.

9.8 Food Safety

La pantalla Food Safety permite buscar alimentos para perros o gatos. La app clasifica los resultados como:

- Seguro.
- Precaucion.
- Toxico.
- Desconocido.

Una decision importante es que la app no inventa respuestas: si no hay informacion suficiente, marca el alimento como desconocido.

9.9 Analisis de productos

La app permite analizar productos por nombre o codigo de barras usando Open Pet Food Facts. Despues cruza los ingredientes encontrados con la base local de alimentos de riesgo.

Esto permite explicar en la demostracion que la app no solo busca alimentos sueltos, sino que tambien revisa productos reales.

9.10 Asistente IA

El asistente IA genera un resumen de cuidados de una mascota. Usa como contexto:

- Datos de la mascota.
- Eventos recientes.
- Notas de salud.
- Recordatorios pendientes.

El resultado incluye:

- Resumen general.
- Prioridades.
- Preguntas para el veterinario.
- Recomendaciones generales.
- Aviso de seguridad.

La IA se usa con limites claros: no diagnostica, no recomienda medicacion ni sustituye al veterinario.

9.11 Emergencia y veterinarios

La app incluye contactos veterinarios y una pantalla de emergencia con senales de alerta, datos utiles y veterinarios marcados como contacto urgente.

9.12 Informes y preparacion de visita

La app tiene pantallas orientadas a preparar una visita veterinaria. Muestran pendientes, notas recientes y preguntas sugeridas. Esto refuerza la idea principal: organizar informacion para tomar mejores decisiones con un profesional.

10. Decisiones tecnicas clave

Flutter como tecnologia principal

Se eligio Flutter porque permite construir una app movil con una unica base de codigo y una interfaz fluida. Tambien facilita crear pantallas de formulario, listados, navegacion y tests de interfaz.

Supabase para backend

Se eligio Supabase porque ofrece autentificacion, base de datos PostgreSQL, politicas de seguridad y una integracion sencilla con Flutter.

Riverpod para estado

Se uso Riverpod para centralizar el estado de la aplicacion. Las pantallas no gestionan directamente toda la logica; observan el estado y llaman al controlador cuando el usuario realiza acciones.

Modelo de datos relacional

La informacion se organiza por usuario, mascota y registros asociados. Esto permite tener relaciones claras: un usuario tiene varias mascotas, y cada mascota puede tener eventos, notas, documentos, pesos y resúmenes.

IA limitada y responsable

La IA no se usa para diagnosticar. Se usa para resumir informacion ya registrada y preparar preguntas para el veterinario. Esta decision reduce riesgos y hace que la funcionalidad sea mas segura.

Modo demo

Se anadio un modo demo para la presentacion. Carga datos preparados como un perro, un gato, vacunas, notas, peso, contacto veterinario, documentos, producto con riesgo y resumen IA. Esto permite grabar un video claro sin depender de crear todos los datos en directo.

11. Alternativas valoradas

Se podrian haber usado otras opciones:

- Firebase en lugar de Supabase.
- Una base local sin backend.
- Una web en lugar de app movil.
- Una IA mas abierta tipo chat libre.

Finalmente se eligio Supabase porque cumple mejor el requisito de base de datos real, autenticacion y seguridad. Tambien se eligio una IA acotada porque es mas apropiada para un contexto sensible como el cuidado de mascotas.

12. Testing y validacion

El proyecto incluye tests para validar el comportamiento de la app.

Se han probado:

- Onboarding inicial.
- Login y registro.
- Validaciones de formulario.
- Creacion de mascotas.
- Eventos y recordatorios.
- Marcado de eventos como completados.
- Notas de salud.
- Busqueda de alimentos.
- Analisis de productos.
- Datos demo para la grabacion.
- Renderizado de pantallas principales.
- Servicios externos con casos de respuesta y fallback.

Comandos de validacion usados:

```
flutter analyze
```

```
flutter test -r expanded
```

```
flutter build web
```

Resultado de la validacion local:

- Analisis sin errores.
- Tests pasados.
- Build web generado correctamente.

13. Mejoras futuras

Algunas mejoras que se podrian anadir son:

- Notificaciones push para recordatorios.
- Subida real de documentos veterinarios.
- Exportacion completa a PDF desde la app.
- Soporte para mas especies.
- Compartir mascotas con familiares.
- Panel para veterinarios.
- Graficas mas avanzadas de peso, sintomas y actividad.
- Escaneo real de codigos de barras con la camara.
- Mejoras en el asistente IA con mas contexto historico.

14. Conclusion

PetCare AI Companion es un proyecto funcional que cumple los requisitos del modulo: desarrolla software real, usa base de datos, trabaja con control de versiones y presenta una solucion coherente a un problema concreto.

La aplicacion no se limita a ser una maqueta visual. Tiene modelos de datos, persistencia, autenticacion, validaciones, pantallas conectadas, tests, integraciones externas y una funcionalidad de IA con limites responsables.

El resultado es un MVP preparado para demostracion academica y tambien para evolucionar como proyecto de portfolio.